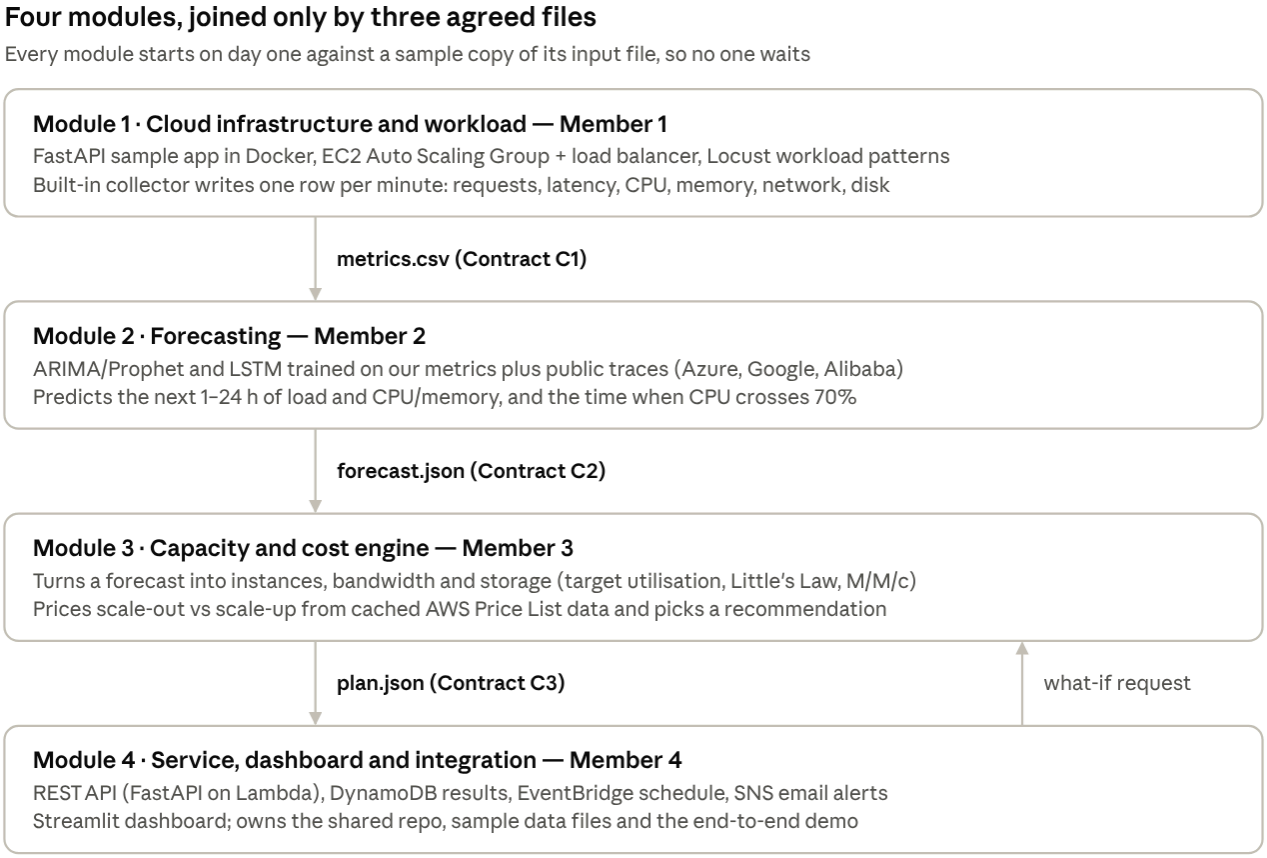
Cloud Cost & Capacity Oracle — Team Work Plan

Oct 23, 2026 · @Interns

Each of the four members owns one module end to end, and the modules talk only through three files with fixed formats (Contracts C1–C3). Everything runs on free tiers: the shared AWS account is on the AWS Free plan, which cannot charge money.

How the work splits



module split · 4 modules, 3 contracts

Data flows downward: Module 1's metrics feed the forecaster, the forecast feeds the capacity and cost engine, and Module 4 serves the resulting plan. Module 4 also sends "what-if" traffic questions straight to Module 3.

Member	Module	Starts today with	Ready for Phase II (Oct 30)
Member 1	Cloud infrastructure and workload	Docker Compose on a laptop	Sample app on EC2, Locust running 2 patterns, real metrics.csv from AWS
Member 2	Forecasting	Public traces + generated sample metrics.csv	ARIMA forecast working, first LSTM trained, accuracy table
Member 3	Capacity and cost engine	Hand-written sample forecast.json	Sizing + cost for scale-out vs scale-up, 500 vs 5,000 req/min example
Member 4	Service, dashboard and integration	Sample plan.json and forecast.json	API returning a plan, Streamlit dashboard showing it, repo set up

Pick roles by strength: Member 1 likes cloud/DevOps, Member 2 likes ML, Member 3 likes maths and Python logic, Member 4 likes web and is organised enough to run integration.

Zero-cost stack

One member creates a new AWS account and picks the **Free plan**. A Free-plan account gets $100 in credits at sign-up and $100 more for five starter activities, cannot be charged until someone chooses to upgrade, and lasts 6 months, which is past Phase III (AWS announcement). The other three get IAM users in that account. Nobody needs AWS to start: every module first runs on a laptop.

Part	Free choice	Why / what changed from the synopsis
Local development	Docker Compose (app × N replicas + nginx load balancer)	Each member can run the full system offline; replicas = horizontal scaling, `--cpus` / `--memory` limits = vertical scaling
Monitored app	EC2 t3.micro (1–5 instances) in an Auto Scaling Group	Small, cheap instance; run only during experiments
Load balancer	Application Load Balancer, created per experiment and deleted after	It bills by the hour, so never leave it running

Part	Free choice	Why / what changed from the synopsis
Metrics	Our own collector (psutil + request middleware) writing to S3	CloudWatch 1-minute and memory metrics are paid; our collector is free and gives exactly the C1 columns
Forecast training	Google Colab (free GPU/CPU)	TensorFlow is too heavy to train in Lambda; train in Colab, save the model to S3
Engine + API	AWS Lambda (container image) + API Gateway	Lambda's monthly free allowance covers a run every 15 min many times over
Results store	DynamoDB (on-demand)	Free allowance covers our few thousand items
Schedule / alerts	EventBridge Scheduler / SNS email	Tiny volumes; within free allowances
Prices	AWS Price List API, cached to a JSON file weekly	The API itself is free; caching keeps the engine fast and offline-testable
Dashboard hosting	Streamlit Community Cloud	Free hosting straight from the GitHub repo
Code + CI	GitHub (private repo) + GitHub Actions	Free for a 4-person team
Public datasets	Azure, Google, Alibaba traces; Wikipedia pageviews	All free downloads

Rules that keep the bill at zero

- Create an **AWS Budgets alert at $1** on day one (it is also one of the five $20 credit activities).
- Everything in AWS is created from scripts in `infra/` so it can be torn down in one command; run `make down` after every session.
- Experiments happen in planned 1–2 hour slots; nothing except Lambda, DynamoDB, S3 and the schedule stays up between them.
- Use one region for everything (ap-south-1, Mumbai) and tag every resource `project=oracle`.

- If the AWS account ever becomes a problem, the Docker Compose setup is the complete fallback demo.

Shared contracts

These formats are frozen on day one and live in `contracts/` with one sample file each. A change needs a message in the team group and agreement from whoever reads that file. Each module is tested against the sample files, so it works with the real data the first time it arrives.

C1 — `metrics.csv` (Module 1 writes → Modules 2 and 3 read). One row per instance per minute. Times in UTC ISO 8601.

```
timestamp,instance_id,instance_type,requests_per_min,avg_latency_ms,p95_laten
cy_ms,cpu_pct,mem_pct,net_in_kb,net_out_kb,disk_used_gb,workload_pattern
2026-10-25T10:00:00Z,i-
01,t3.micro,512,41.2,88.0,34.5,52.1,820,6400,3.1,steady
2026-10-25T10:01:00Z,i-
01,t3.micro,498,39.8,85.4,33.9,52.3,801,6230,3.1,steady
```

C2 — `forecast.json` (Module 2 writes → Module 3 reads, Module 4 displays).

```
{
  "generated_at": "2026-10-25T10:15:00Z",
  "model": "lstm",
  "horizon_minutes": 1440,
  "step_minutes": 15,
  "accuracy": {"mape_pct": 8.4, "rmse": 42.1},
  "series": [
    {"timestamp": "2026-10-25T10:30:00Z", "requests_per_min": 640, "cpu_pct":
41.0, "mem_pct": 55.0, "lower": 590, "upper": 700}
  ],
  "cpu_threshold_pct": 70,
  "exhaustion_at": "2026-10-25T18:45:00Z"
}
```

C3 — `plan.json` (Module 3 writes → Module 4 stores and serves).

```
{
  "generated_at": "2026-10-25T10:15:00Z",
  "peak_requests_per_min": 5000,
  "current": {"instances": 1, "instance_type": "t3.micro", "usd_per_hour":
0.0112},
  "options": [
    {"strategy": "scale_out", "instances": 6, "instance_type": "t3.micro",
"usd_per_hour": 0.067, "usd_per_day": 1.61, "expected_latency_ms": 52,
"single_point_of_failure": false},
    {"strategy": "scale_up", "instances": 1, "instance_type": "t3.xlarge",
"usd_per_hour": 0.179, "usd_per_day": 4.30, "expected_latency_ms": 47,
"single_point_of_failure": true}
  ],
  "bandwidth_mbps": 4.3,
  "storage_gb_30d": 5.2,
  "recommendation": "scale_out",
  "reason": "62% cheaper per day and no single point of failure; latency
within 10% of scale-up",
  "exhaustion_at": "2026-10-25T18:45:00Z"
}
```


The prices in these samples are placeholders for testing; Module 3 fills real ones from the Price List cache.

API (Module 4 serves).

Method	Path	Returns
GET	`/forecast/latest`	Latest C2
GET	`/plan/latest`	Latest C3
POST	`/plan` with `{"expected_rpm": 5000}`	A fresh C3 for that what-if load (calls Module 3 directly)
GET	`/history?days=7`	Past plans, for the dashboard
GET	`/health`	`{"status": "ok"}`

Modules 2 and 3 are **plain Python packages** with one function each — `forecast(metrics_df) -> dict` and `plan(forecast: dict, baseline: dict) -> dict` — so Module 4 can import them into Lambda without any web calls between modules.

Member tasks

Everyone, by Sat Oct 24 (one 1-hour call)

- [ ] Agree who is Member 1–4 and read the contracts section together
- [ ] Member 4 creates the GitHub repo with the folder layout below; everyone clones it
- [ ] Member 1 creates the AWS Free-plan account, the $1 budget alert and three IAM users
- [ ] Each member commits a stub of their module that reads the sample input and writes a sample output

Member 1 — Cloud infrastructure and workload

Phase II (by Oct 29)

- [ ] FastAPI sample app with 3 endpoints of different weight (light page, CPU-heavy calculation, large download) in Docker
- [ ] Collector inside the app: middleware for request count and latency, psutil for CPU/memory/network/disk; writes C1 rows every minute, uploads to S3 every 5 min
- [ ] Docker Compose: app × 3 + nginx, for local runs
- [ ] Locust scripts for 4 patterns: steady 500 req/min, daily cycle, 10× spike, steady growth
- [ ] One AWS run: 1 EC2 t3.micro, steady + spike patterns, 2 hours of real metrics.csv in S3

Phase III

- [ ] `infra/` scripts (AWS CLI or Terraform) for Auto Scaling Group + ALB, with `make up` / `make down`
- [ ] Vertical runs (t3.micro → t3.small → t3.medium) and horizontal runs (1 → 5 instances) under the same load, to measure real capacity per instance type
- [ ] Reactive-baseline run with plain AWS target-tracking autoscaling, for comparison with the Oracle's advice

Member 2 — Forecasting

Phase II (by Oct 29)

- [ ] Loaders that turn the Azure VM trace and Wikipedia pageviews into C1-shaped data
- [ ] Generator for synthetic C1 data in the 4 patterns (used by everyone until real data arrives)
- [ ] ARIMA (statsmodels) forecast with MAE/RMSE/MAPE on a held-out day; `forecast()` returns C2 including `exhaustion_at`
- [ ] First LSTM trained in Colab on one trace

Phase III

- [ ] Prophet + tuned LSTM; accuracy table for every model × pattern × horizon (1 h, 6 h, 24 h)
- [ ] Pick the model per pattern automatically from recent error
- [ ] Retrain on our own AWS metrics; save model to S3 for Lambda to load

Member 3 — Capacity and cost engine

Phase II (by Oct 29)

- [ ] `baseline(metrics_df)`: CPU % per 100 req/min, KB per request, storage growth per day
- [ ] `plan()` sizing: instances = ceil(peak ÷ capacity at 70% CPU) + 1; bandwidth; storage
- [ ] Price List API script that saves on-demand Linux prices for t3/t4g/m6i/c6i in ap-south-1 to `pricing.json`
- [ ] Scale-out vs scale-up comparison and the worked 500 → 5,000 req/min example

Phase III

- [ ] M/M/c response-time estimate and Little's Law concurrency check, validated against Member 1's measured latency
- [ ] Measured capacity per instance type (from Member 1's runs) replaces assumptions
- [ ] Unit tests for every formula; sensitivity table (threshold 60/70/80%)

Member 4 — Service, dashboard and integration

Phase II (by Oct 29)

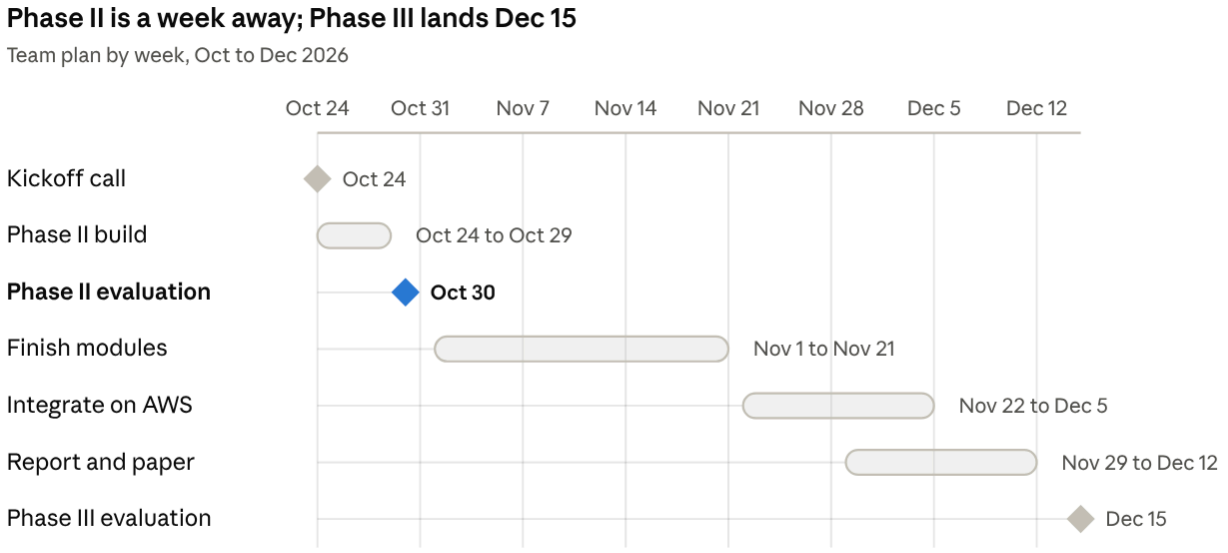
- [ ] Repo, README, `contracts/` sample files, GitHub Actions running everyone's tests
- [ ] FastAPI with the 5 endpoints, first serving sample files, then calling Modules 2 and 3
- [ ] Streamlit dashboard: forecast vs actual chart, exhaustion countdown, scale-out vs scale-up cost cards, what-if box
- [ ] Phase II slide deck: collect one demo screenshot and one result from each member

Phase III

- [ ] Deploy API + engine to Lambda (container image) behind API Gateway; DynamoDB tables `forecasts` and `plans`
- [ ] EventBridge schedule every 15 min; SNS email when `exhaustion_at` is within 60 min
- [ ] Streamlit Community Cloud deployment; recorded end-to-end demo video as a backup
- [ ] Assemble the final report and paper from everyone's sections

Report and paper split (Phase III): each member writes the methodology and results for their own module; Member 2 also writes the literature review, Member 4 writes introduction, architecture and conclusion.

Timeline



timeline · Oct 24 to Dec 15, 2026

Phase II (Oct 30–31) needs each module working on its own plus one real AWS run, not full integration. Integration on AWS happens in the two weeks after modules are finished, leaving a buffer before Phase III (Dec 15–20).

What to show at Phase II (rubric: objectives and methodology, system design, presentation — 5 marks each): the architecture slide, Member 1's live app under a Locust spike, Member 2's forecast vs actual chart with MAPE, Member 3's 500 → 5,000 req/min plan with scale-out vs scale-up costs, and Member 4's dashboard showing that plan.

Repo layout and working rules

```
oracle/
├── contracts/          # C1-C3 sample files + schema notes (Member 4,
everyone reviews)
├── workload/           # Module 1: app/, collector/, locust/, docker-
compose.yml
├── infra/              # Module 1: AWS scripts, Makefile (up / down)
├── forecasting/        # Module 2: loaders, models, notebooks/, tests/
├── capacity/           # Module 3: baseline, sizing, pricing, pricing.json,
tests/
├── service/            # Module 4: api/, lambda/, dashboard/
├── data/               # small samples only; real data stays in S3
└── docs/               # report, paper, diagrams
```

- **Own your folder.** Only touch another module's folder through a pull request that its owner approves.

- **Branches:** `m1-*`, `m2-*`, `m3-*`, `m4-*`; merge to `main` through pull requests; `main` must always pass tests.
- **Integration days:** Tue and Sat, 30 minutes — Member 4 runs the full pipeline on the latest `main` and lists what broke.
- **Daily update:** one line each in the team group — done, next, blocked.
- **Secrets:** AWS keys only in local `.env` files and GitHub Actions secrets, never in the repo.
- **Python 3.11** for everyone; each module keeps its own `requirements.txt`.